Proyecto 2 | Entrega 1

Pipeline RV32I de cinco etapas e inicio Dual-Issue

Campo	Completar antes de Canvas
Integrantes y códigos	[LISTA COMPLETA DE INTEGRANTES Y CÓDIGOS]
Curso, sección, docente	[CURSO / SECCIÓN / DOCENTE]
Fecha de entrega	[FECHA CONFIRMADA DE E1]
Otros requisitos	[ESPACIO PARA INDICACIONES ADICIONALES DEL CURSO]

Alcance y estado verificable

El procesador A ejecuta las cinco etapas IF, ID, EX, MEM y WB. La regresión automatizada termina con siete verificaciones PASS: P1-P4 exigidos, Dual Fetch/Decode, Register File doble aislado y una prueba adicional de ISA. El prototipo B de E1 es independiente del top A; todavía no es el procesador Dual-Issue completo de E2.

ISA implementada en A: 37 instrucciones de cálculo, transferencia y control del RV32I base: LUI, AUIPC, JAL, JALR; BEQ/BNE/BLT/BGE/BLTU/BGEU; LB/LH/LW/LBU/LHU; SB/SH/SW; ADDI/SLTI/SLTIU/XORI/ORI/ANDI/SLLI/SRLI/SRAI; ADD/SUB/SLL/SLT/SLTU/XOR/SRL/SRA/OR/AND. La memoria de datos usa un puerto y direcciones de byte. No se implementan excepciones, ECALL ni EBREAK; FENCE se trata como NOP en este sistema mononúcleo sin periféricos. Los accesos de medio word y word de la prueba son alineados.

Resultado de las pruebas

Prueba	Evidencia	Resultado
P1	ISA sin hazards; 15 instrucciones del subconjunto inicial	PASS; 72 ciclos; 0 stalls
P2	RAW, forwarding MEM/WB y hacia store	PASS; 11 ciclos; 6 ciclos con forwarding
P3	Load-Use y burbuja	PASS; 9 ciclos; 1 stall
P4	BEQ/JAL y anulación de camino incorrecto	PASS; 13 ciclos; 2 redirects
ISA extra	Operaciones restantes, subwords, branches, JALR	PASS; 63 ciclos
B preliminar	Dos PC/instrucciones, Decode y dos ALU	PASS
RF doble	Cuatro lecturas, dos escrituras, x0 y prioridad	PASS

Arquitectura A: pipeline y control

IF lee una instrucción y calcula PC+4. IF/ID guarda instrucción, PC y validez. ID decodifica opcode/funct3/funct7, lee el Register File y extiende inmediatos I, S, B, J y U. ID/EX propaga operandos, destino y controles. EX calcula ALU, evalúa branches y decide el siguiente PC. EX/MEM conserva dirección, dato y tipo de acceso; MEM hace un acceso de datos. MEM/WB selecciona ALU, carga o PC+4 y escribe el destino en el flanco de bajada.

FIGURA A — DATAPATH SINGLE-ISSUE [PENDIENTE DEL EQUIPO]

Insertar diagrama elaborado por los compañeros: IF/ID, ID/EX, EX/MEM, MEM/WB; Controller; Hazard Unit; multiplexores de forwarding MEM/WB; stall de PC e IF/ID; burbuja y flush; camino de memoria y writeback. Citar número de figura y autoría final.

Hazard Unit, forwarding, stall y flush

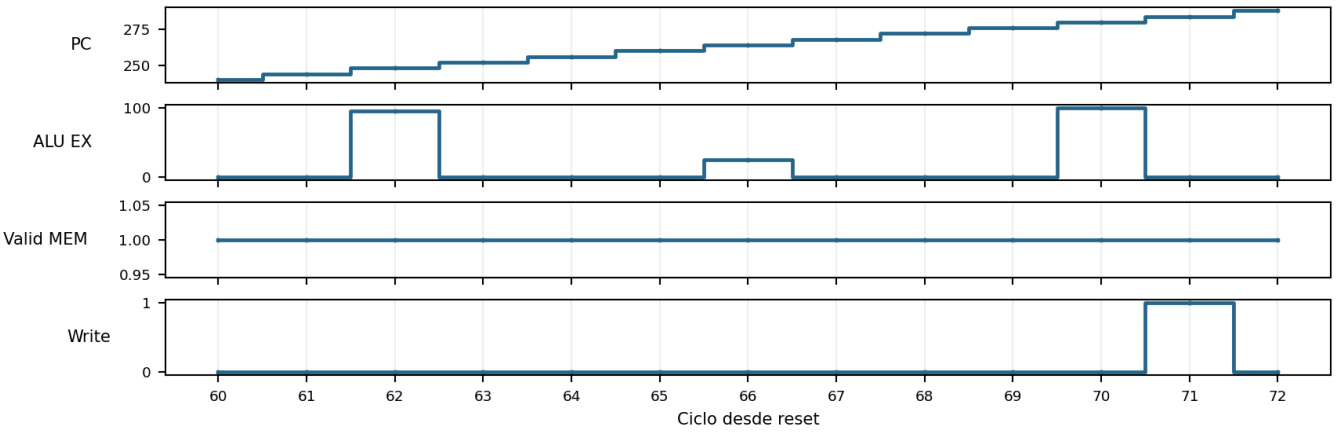
Las fuentes rs1/rs2 de EX comparan con los destinos válidos en MEM y WB; MEM tiene prioridad si ambos escriben el mismo registro. Un load en MEM no puede reenviarse desde la ALU. Un load en EX seguido por un consumidor en ID retiene PC e IF/ID un ciclo e inserta una burbuja en ID/EX. Un branch tomado o JAL/JALR redirige el PC en EX y anula IF/ID e ID/EX. Los bits Valid impiden escrituras de instrucciones anuladas. El dato de store también recibe forwarding.

Decodificación y recursos

Clase	Control principal en EX/MEM/WB
ALU R/I	ALU con operandos RF o inmediato; resultado a WB
LUI/AUIPC	Inmediato U; fuente A = cero o PC
Load/store	Dirección byte por ALU; funct3 decide ancho y signo
Branch	Comparación signed/unsigned de operandos reenviados; destino PC+offset
JAL/JALR	Redirección; WB recibe PC+4; JALR limpia bit 0 del destino

P1 — ISA sin dependencias que requieran Hazard Unit

El programa separa productores y consumidores con tres NOP. Comprueba ADDI, ADD, SUB, SLT, AND, OR, SLL, SRL, SLTI, ANDI, ORI, SLLI, SRLI, LW y SW. El testbench compara resultados en registros y memoria; no basta con que el programa termine.



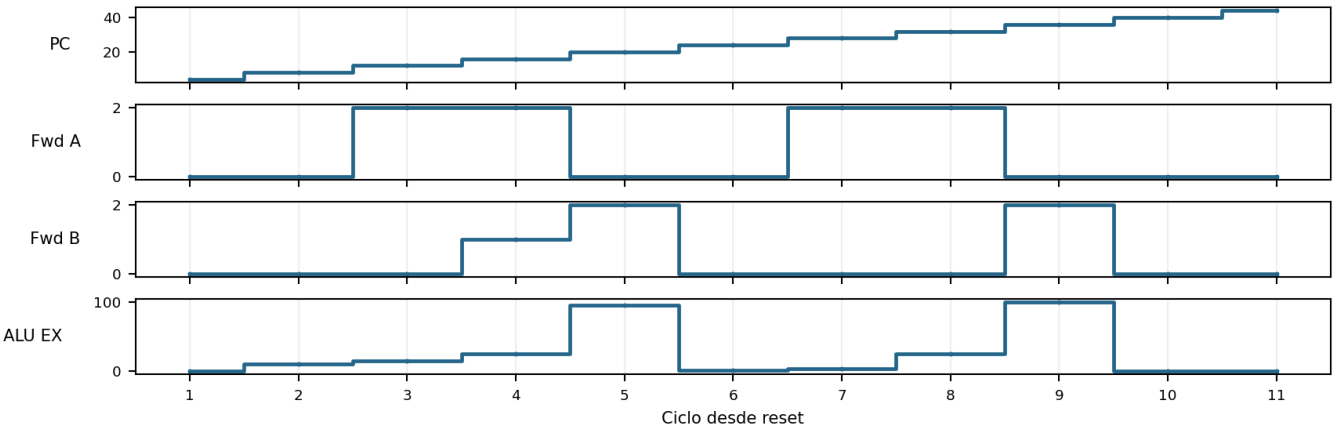
Últimos ciclos: el PC avanza sin retención; el store final aparece una sola vez y Stall_D permanece en cero.

Resultado intermedio/final	Esperado	Observado
x3 = x1+x2	25	25
x4 = x2-x1	5	5
x8 = x5 << x1	1024	1024
x9 = x8 >> x5	512	512
mem[96] tras SW	10	10
x10 tras LW; x11; mem[100] final	10; 25; 25	10; 25; 25

Comando: run_e1.ps1, caso 1. Fuente: tests/e1_1.S y tests/e1_1.mem. Waveform completa: tests/e1_1.vcd. Resultado: PASS, 72 ciclos, 0 stalls.

P2 — Forwarding entre etapas

Secuencia decisiva: ADDI x1=10, ADDI x2=x1+5, ADD x3=x2+x1, SW x3. Luego dos productores consecutivos de x4 obligan a escoger el valor más reciente. Fwd=2 significa MEM, Fwd=1 significa WB.



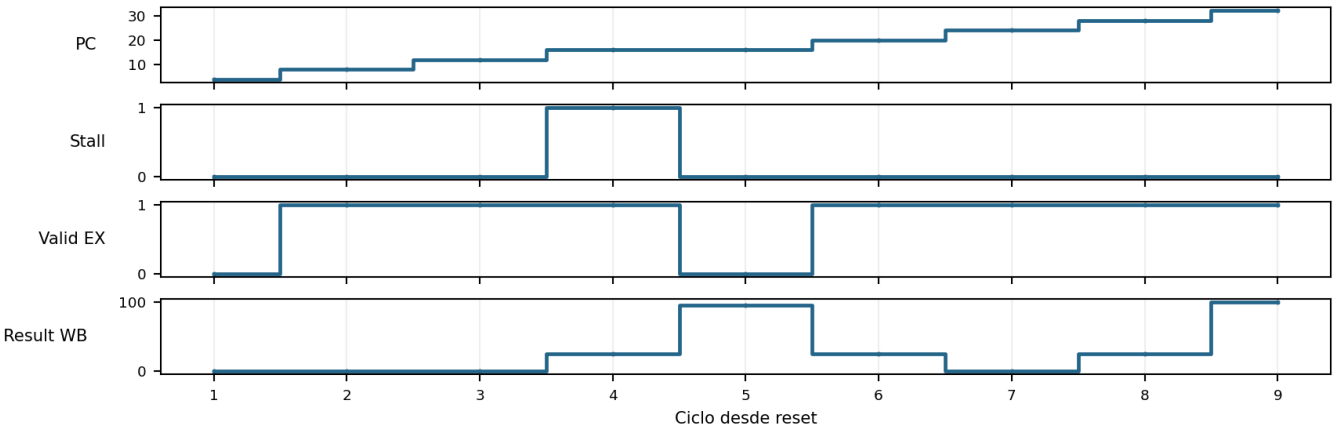
Valores tomados de tests/e1_2.vcd al final de cada ciclo. En ciclo 4, x2=15 llega desde MEM y x1=10 desde WB; la ALU produce 25. En ciclo 7 se selecciona el productor reciente de x4 y se obtiene 3.

Ciclo	Fwd A/B	SrcA EX	B reenviado	ALU EX
3	2/0	10	X	15
4	2/1	15	10	25
5	0/2	0	25	96
7	2/0	1	15	3
8	2/0	3	X	25
9	0/2	0	25	100

El store final escribe 25 en dirección 100. PASS, 11 ciclos, seis ciclos con forwarding y cero stalls. Fuente: tests/e1_2.S/.mem; testbenches/testbench.v.

P3 — Stall Load-Use y burbuja

Secuencia: SW 25 a dirección 96; LW x2 desde 96; ADD x3=x2+x0 inmediatamente después. El dato del load no está disponible mientras la instrucción está en EX.



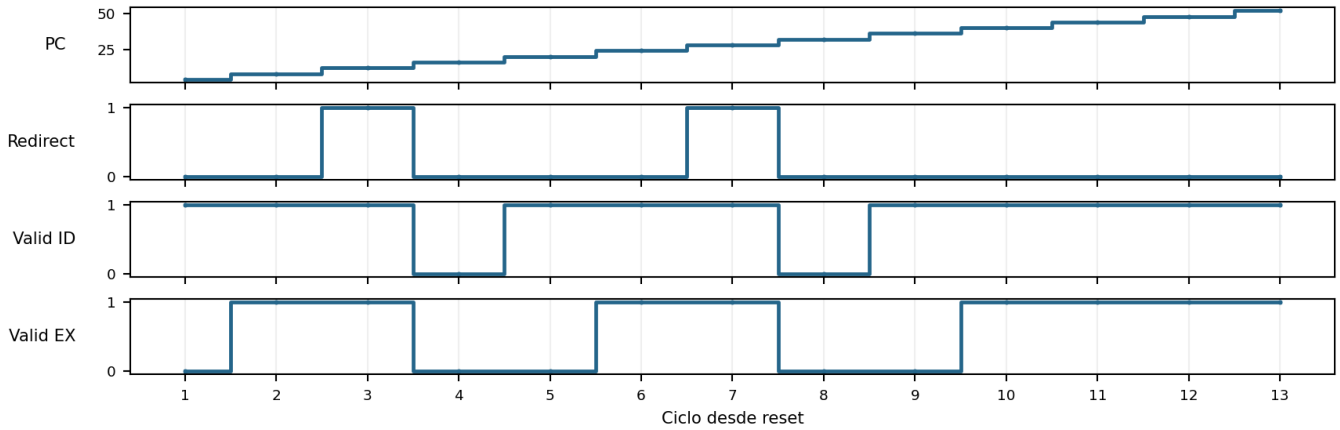
En ciclo 4 Stall_D=1. En el siguiente, PC permanece en 16 y Valid_E=0: se insertó la burbuja sin perder el consumidor. En ciclo 6 WB presenta 25 y el consumidor continúa.

Ciclo	PC IF	Stall	Valid EX	Result WB
4	16	1	1	25
5	16	0	0	96
6	20	0	1	25
8	28	0	1	25

Esperado/observado: memoria[96]=25, x2=25, memoria[100]=25. PASS, 9 ciclos, exactamente un stall. Fuente: tests/e1_3.S/.mem/.vcd; testbenches/testbench.v.

P4 — Flush por cambio de control

El BEQ tomado en PC=4 redirige a PC=16; JAL en PC=20 redirige a PC=32 y escribe PC+4=24 en x3. El BEQ de PC=32 no se toma. Dos SW colocados en el camino incorrecto intentan escribir la dirección 104.



Los redirects de los ciclos 3 y 7 se siguen de Valid ID/EX=0. Así se anulan las instrucciones jóvenes antes de que produzcan efectos.

Ciclo	PC IF	Redirect	PC destino	Valid ID/EX
3	12	1	16	1/1
4	16	0	0	0/0
7	28	1	32	1/1
8	32	0	0	0/0
10	40	0	40	1/1

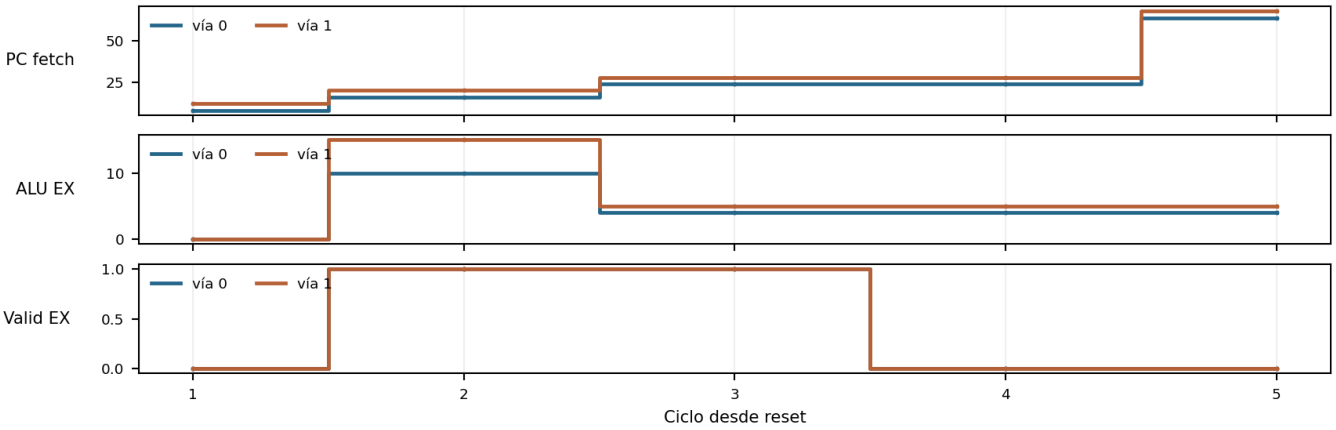
Esperado/observado: ningún store a 104; x3=24; store final 25 en 100. PASS, 13 ciclos y dos redirecciones. Fuente: tests/e1_4.S/.mem/.vcd; testbenches/testbench.v.

B preliminar — Dual Fetch, Dual Decode y segunda vía

imem entrega palabras en PC y PC+4, y el siguiente fetch avanza ocho bytes. Dos Controller decodifican en paralelo y un Register File de cuatro lecturas alimenta dos rutas ID/EX y dos ALU. El control hold conserva el par en ID e inserta una burbuja en EX; flush redirige el PC y anula ambas vías. El Register File de dos escrituras se prueba por separado; sus puertos de escritura aún están deshabilitados en el prototipo B. La Issue Unit, memoria/branch compartidos y WB doble corresponden a E2.

FIGURA B — DATAPATH DUAL-ISSUE PRELIMINAR [PENDIENTE DEL EQUIPO]

Insertar diagrama elaborado por los compañeros: PC0/PC1, dos lecturas de imem, dos decoders, cuatro lecturas del Register File, registros IF/ID e ID/EX por vía y dos ALU. Distinguir gráficamente lo implementado en E1 de lo reservado para E2.



PC0/PC1 = 0/4 y luego 8/12; las instrucciones decodificadas son ADDI de 10/15 y 4/5. Los resultados EX son 10/15 y 4/5. La tabla registra palabras de instrucción y control ALU=0 (ADD) para demostrar Decode.

Ciclo	Instrucción 0/1 en ID	Control ALU 0/1	Resultados EX 0/1
1	0x00A00093 / 0x00F00113	0/0	0/0
2	0x00400193 / 0x00500213	0/0	10/15
3	0x00000013 / 0x00000013	0/0	4/5

Pruebas adicionales: hold mantiene el par; flush lleva PC a 64/68; regfile_dual_tb valida cuatro lecturas, dos escrituras, x0 fijo y prioridad de la vía joven. Ambas verificaciones terminan en PASS.

Ampliación ISA y verificación adicional

La prueba e1_isa_ext no reemplaza P1-P4. Recorre las instrucciones enteras restantes del alcance indicado en la portada y comprueba el estado final de registros y memoria. Incluye branches tomados y no tomados, JALR con bit 0 limpiado, LUI/AUIPC, comparaciones signed/unsigned, XOR y shift aritmético.

Componente	Esperado y observado
SLTU/SLTIU	x3=1; x4=0
XOR/XORI y SRA/SRAI	x5=x6=0xFFFFFFFE; x7=x8=0xFFFFFFFF
LUI/AUIPC	x9=0x12345000; x10=36 (PC de AUIPC)
SB/SH + LB/LBU/LH/LHU/LW	mem[128]=0xFF808000; x13=0xFFFFFF80; x14=0x80; x15=0xFFFFFF80; x16=0xFF80; x17=0xFF808000
Branches y JALR	x19-x29=1; x30=209; JALR salta a 208; x31=204; store final mem[124]=1

Resultado: PASS en 63 ciclos. Los accesos byte y halfword usan dirección de byte dentro de una palabra de 32 bits; las cargas aplican extensión de signo o cero según funct3. Fuente: tests/e1_isa_ext.S/.mem/.vcd y testbenches/isa_ext_tb.v.

Reproducción y paquete para Canvas

En Windows, desde la carpeta pipeline, ejecutar **./run_e1.ps1** con Python e Icarus Verilog en PATH. El script regenera las memorias desde tests/gen_programs.py, compila tres testbenches principales más la prueba de ISA y termina con PASS o error. Los VCD quedan en tests/. Las figuras se trazaron a partir de esas señales registradas por vvp durante la simulación.

Subir los .v, tests/e1_1-4.S y .mem, tests/e1_dual.S/.mem, tests/e1_isa_ext.S/.mem, los VCD, run_e1.ps1, tests/gen_programs.py, README.md y el informe editable. Los binarios .out, tmp/ y tools/ son locales.

Espacio para ajustes del equipo

REQUISITOS ADICIONALES / REVISIÓN FINAL [PENDIENTE]

Anotar aquí indicaciones posteriores del profesor, referencia de las figuras A y B, nombres definitivos, fecha real de sesión 13.1 y revisión del alcance ISA. No presentar el PDF con estos campos sin completar.

Fuentes: Proyecto2_Enunciado.pdf, sección 4.1, y Proyecto2_Indicaciones_Generales_Rubrica.pdf, sección 2. Los ciclos se cuentan desde reset hasta el store de éxito; no constituyen todavía una comparación IPC de A/B/C.